RIDIP design diagrams

Rwanda Integrated Data Intelligence Platform
NISR 2026 Big Data Hackathon

Nine views of the platform, from how people move through it to how its data is stored.
Version 1, 2 October 2026.

Contents

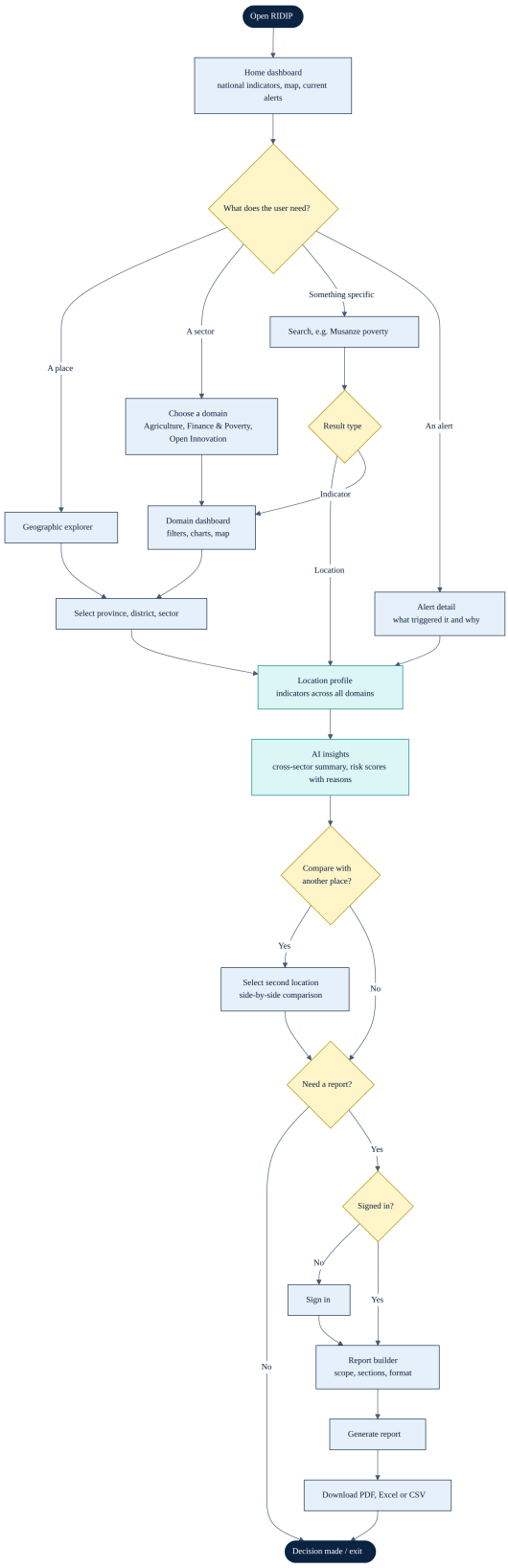
Figure 1	User flow	2
Figure 2	Wireflows	3
Figure 3a	Use case diagrams: Public, analyst and institution users	4
Figure 3b	Use case diagrams: Administration and data management	5
Figure 4	Component and layered architecture	6
Figure 5a	Web application life cycle: Project phases	7
Figure 5b	Web application life cycle: Every change: continuous integration and delivery	8
Figure 6	Navigation graph	9
Figure 7	System architecture	10
Figure 8a	Sequence diagrams: Open a location profile with AI insights	11
Figure 8b	Sequence diagrams: Generate and download a report	12
Figure 9a	Local and remote entity relationship diagrams: Remote: server database (PostgreSQL + PostGIS)	13
Figure 9b	Local and remote entity relationship diagrams: Local: browser cache (IndexedDB)	14

The local cache design and the single indicator-values table are recommendations introduced in these diagrams; all other content follows the RIDIP Project Structure and Implementation Blueprint.

Figure 1

User flow

How a planner, researcher or citizen moves from opening RIDIP to a decision or a downloaded report. Diamonds are decisions; the two teal steps are where every path converges.

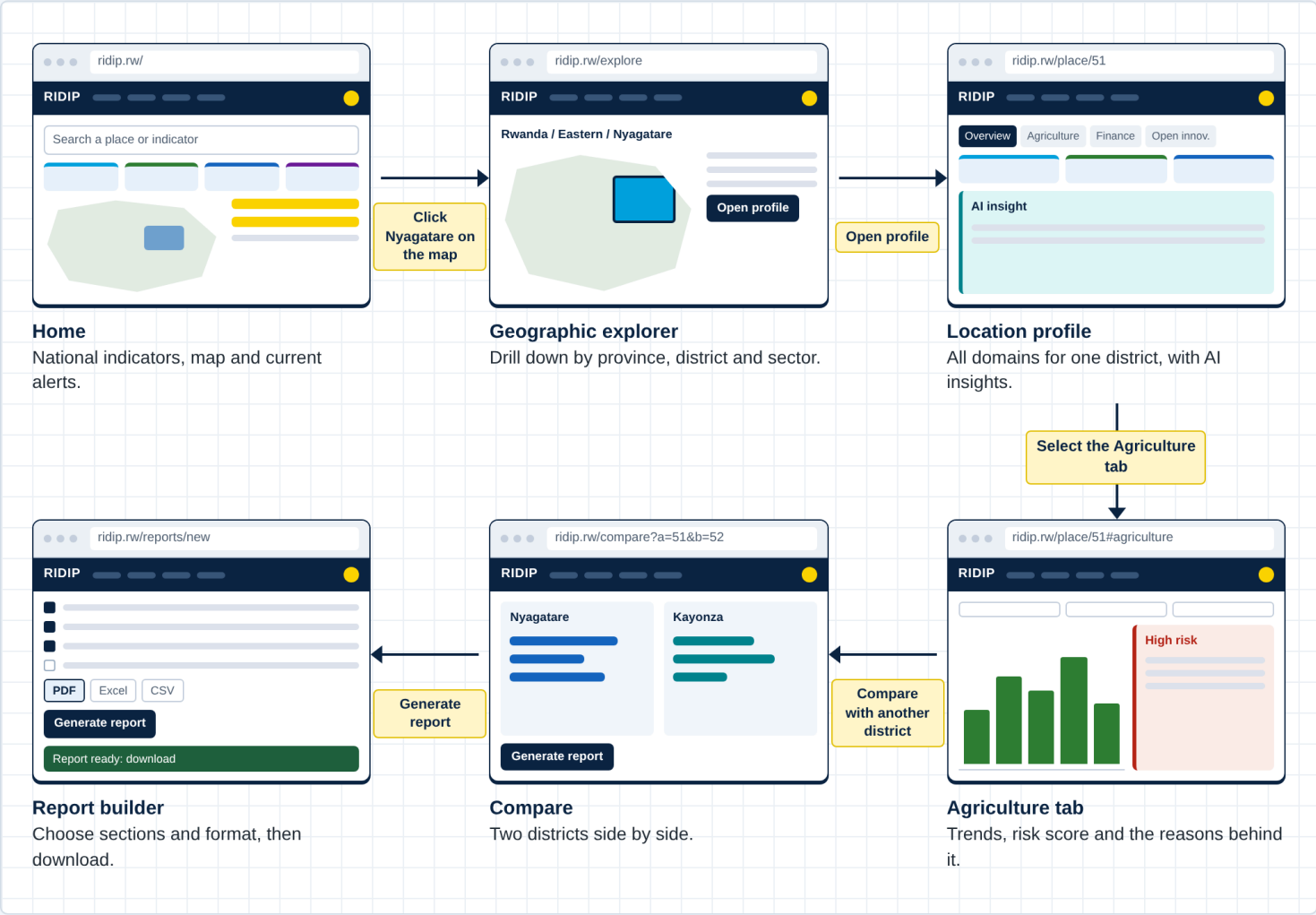


- Four entry points (place, sector, search, alert) all lead to the location profile, so that page carries most of the product’s value.
- Sign-in is only required at the point of exporting, which keeps public exploration frictionless.

Figure 2

Wireflows

Low-fidelity screens joined by the action that moves the user between them. This is the main demo journey: one district, followed across every domain to a finished report.



- Layout is indicative, not final visual design. Each screen keeps the same top navigation so users never lose their place.
- The search box on Home is a shortcut straight to the district profile, skipping the explorer.

Figure 3a

Use case diagrams

Public, analyst and institution users

What each type of user can do in RIDIP. Split into two views so the public-facing features and the administration features each stay readable.

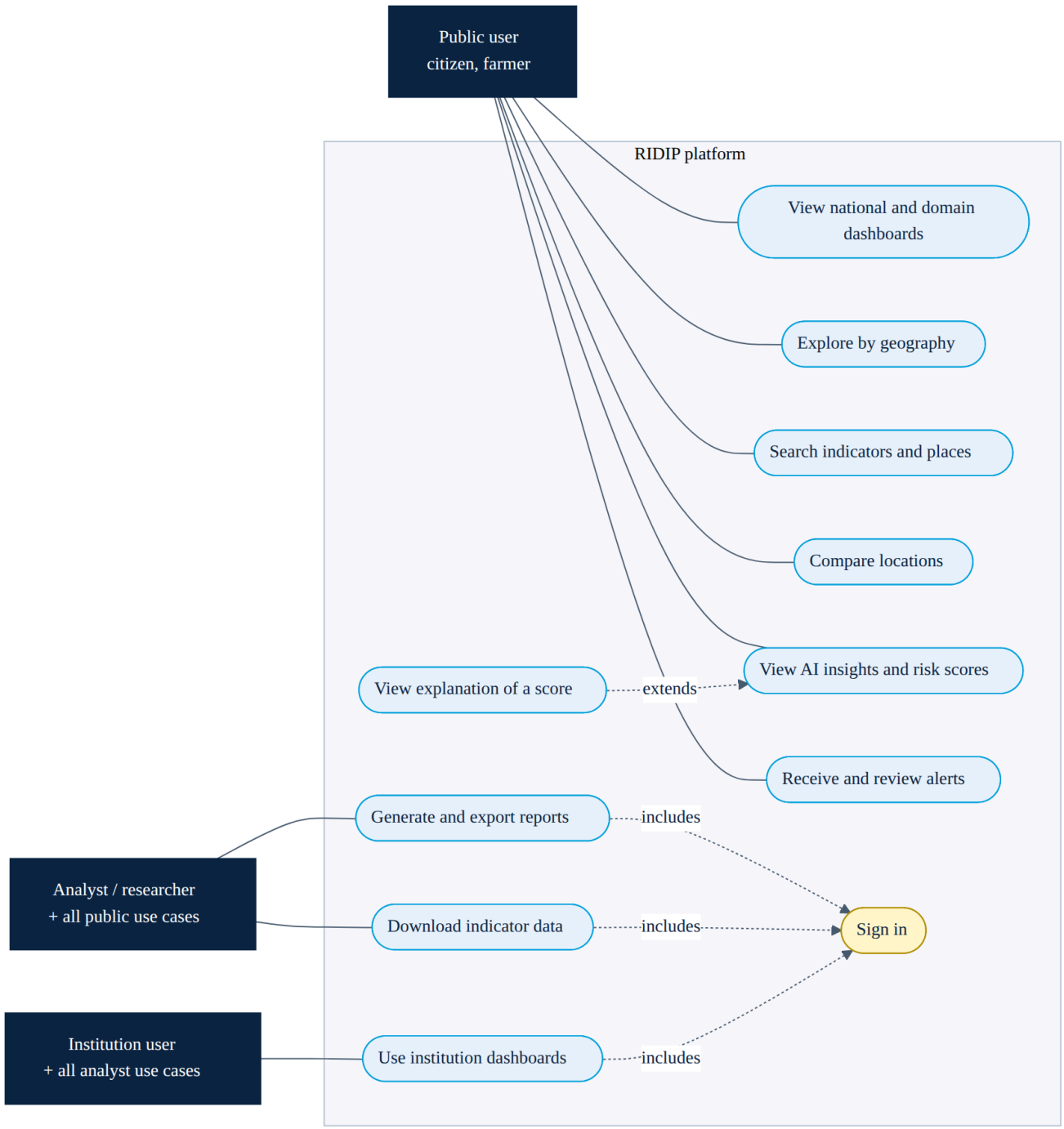
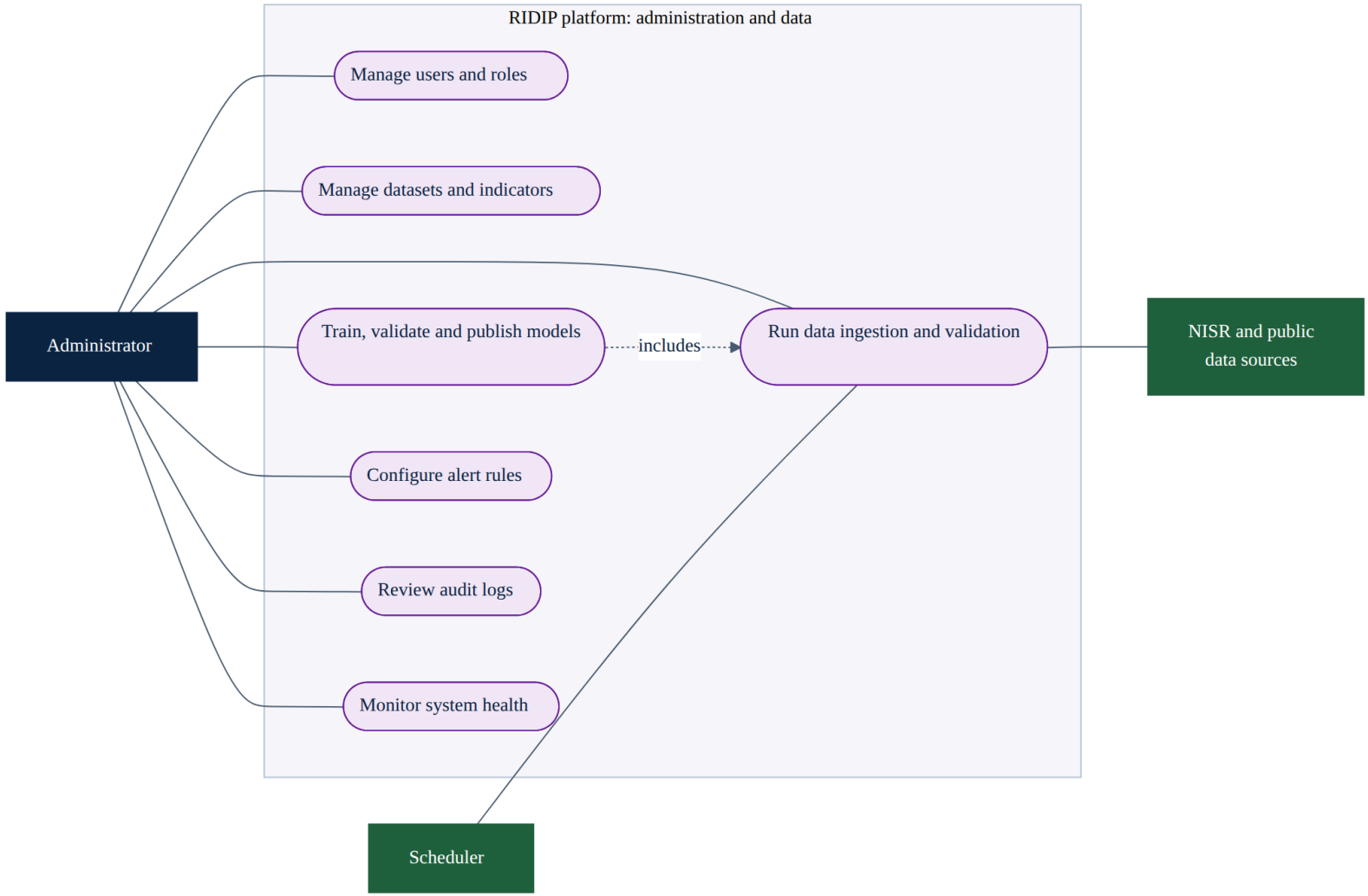


Figure 3b

Use case diagrams

Administration and data management

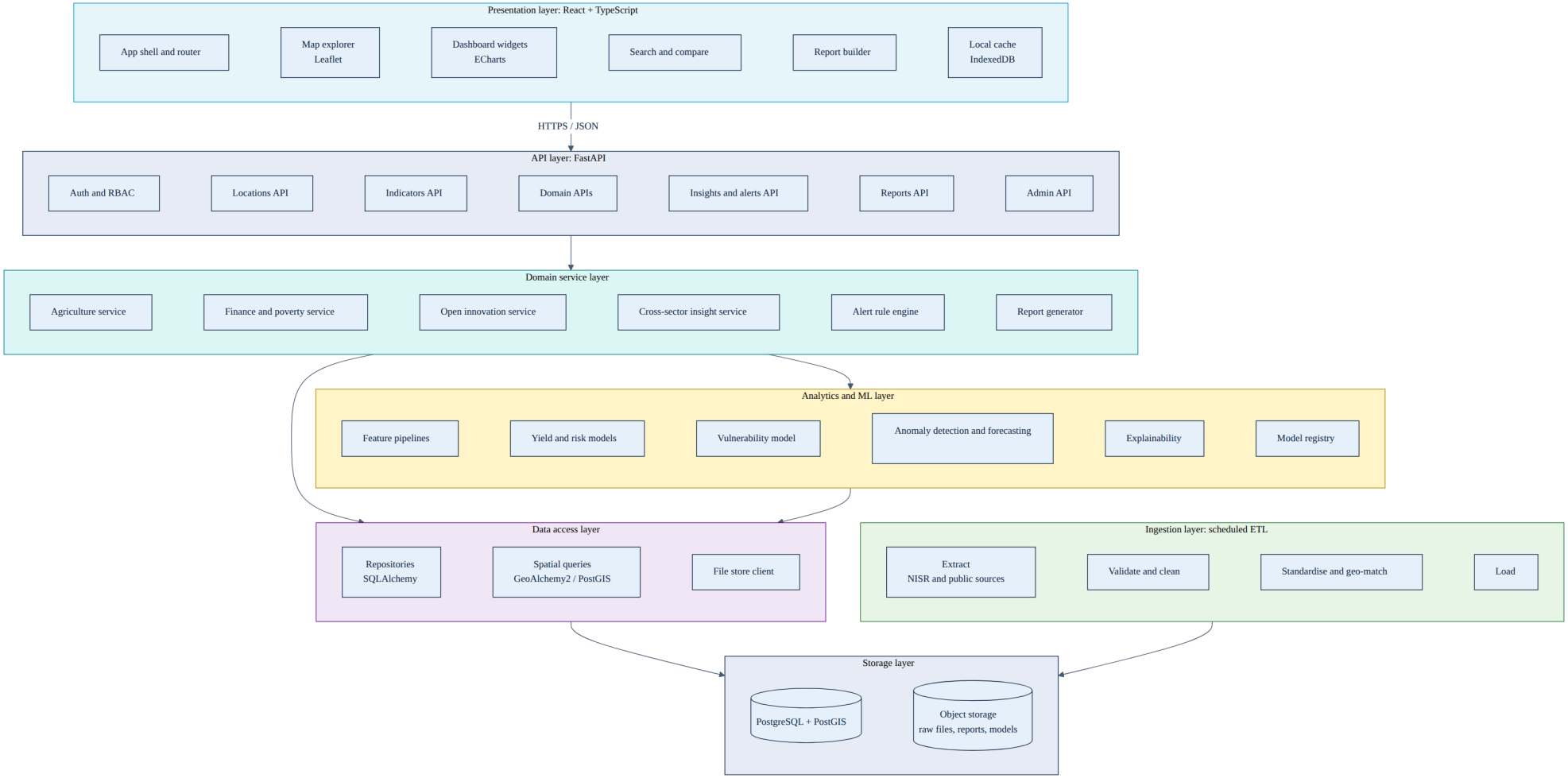


- Roles are cumulative: an analyst can do everything a public user can, and an institution user everything an analyst can.
- “Includes” means the use case always triggers the other one; “extends” means it optionally adds to it.

Figure 4

Component and layered architecture

The software components grouped into layers. Each layer only calls the layer below it, which keeps the codebase easy to split between two developers and to test.



- Domain services hold the business rules; the API layer stays thin and only handles routing, validation and permissions.
- Ingestion runs separately on a schedule and writes directly to storage, so slow downloads never affect users.

Figure 5a

Web application life cycle

Project phases

The development life cycle mapped onto the competition calendar, followed by the continuous integration loop that every code change passes through.

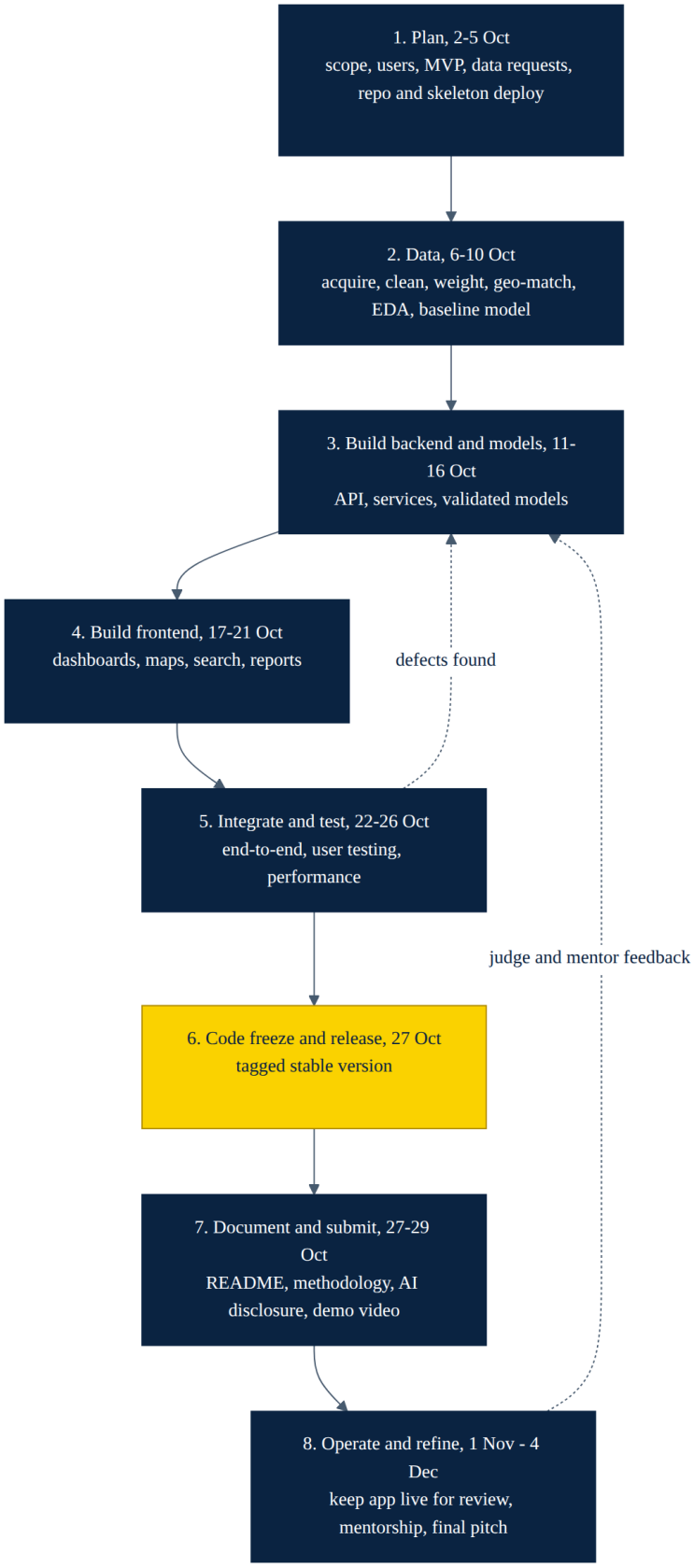
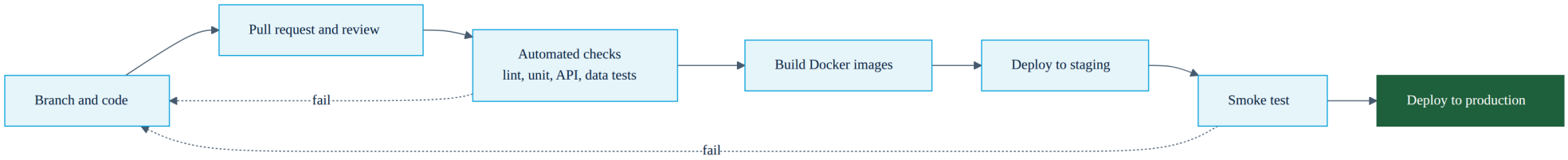


Figure 5b

Web application life cycle

Every change: continuous integration and delivery

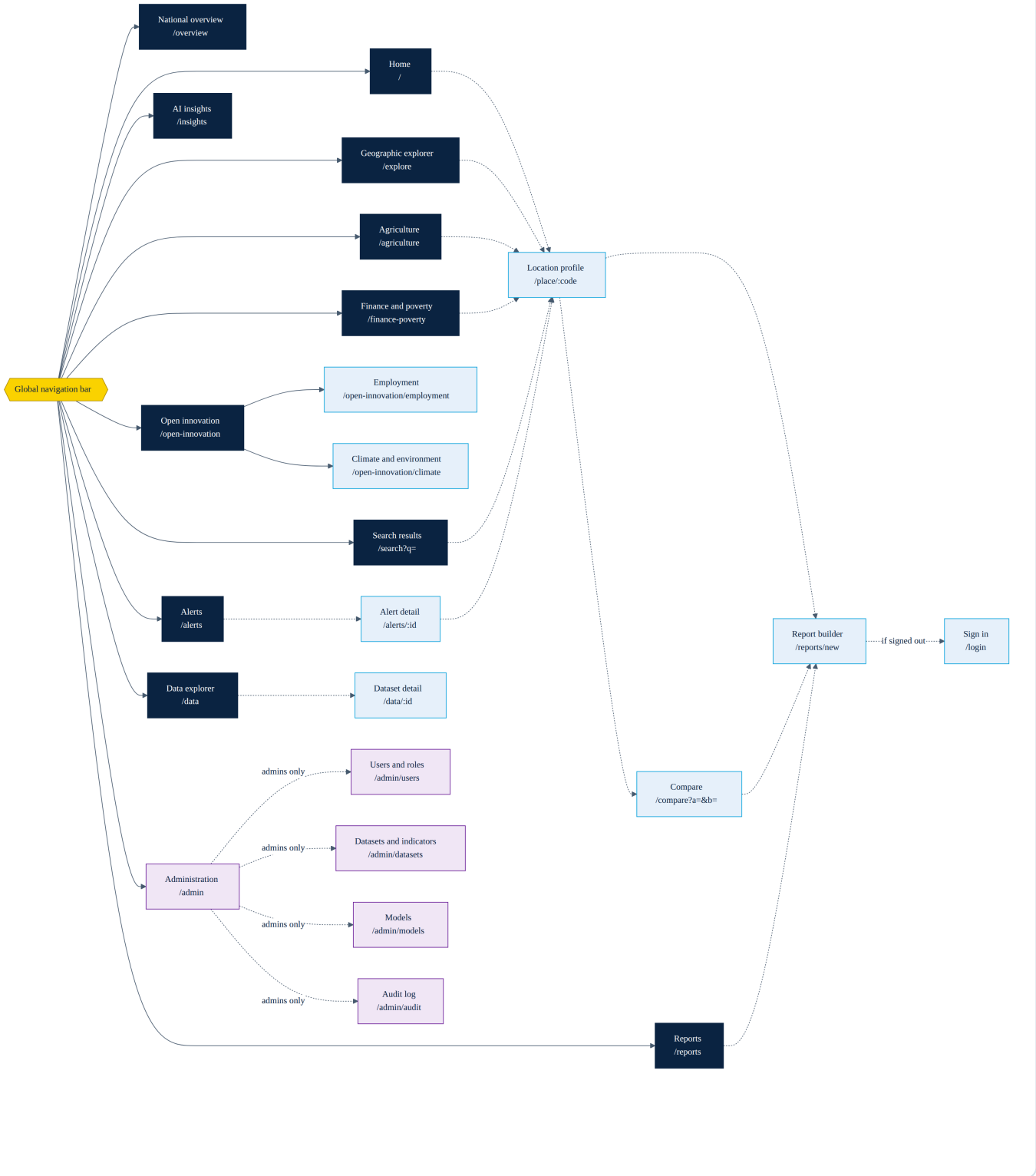


- The code freeze on 27 October is the hard gate: after it, only fixes and documentation.
- The app must stay live through the review period and, if shortlisted, until the final presentations on 4 December.

Figure 6

Navigation graph

Every page in the application and how users reach it. Solid lines are permanent links in the top navigation; dotted lines are links from inside a page.

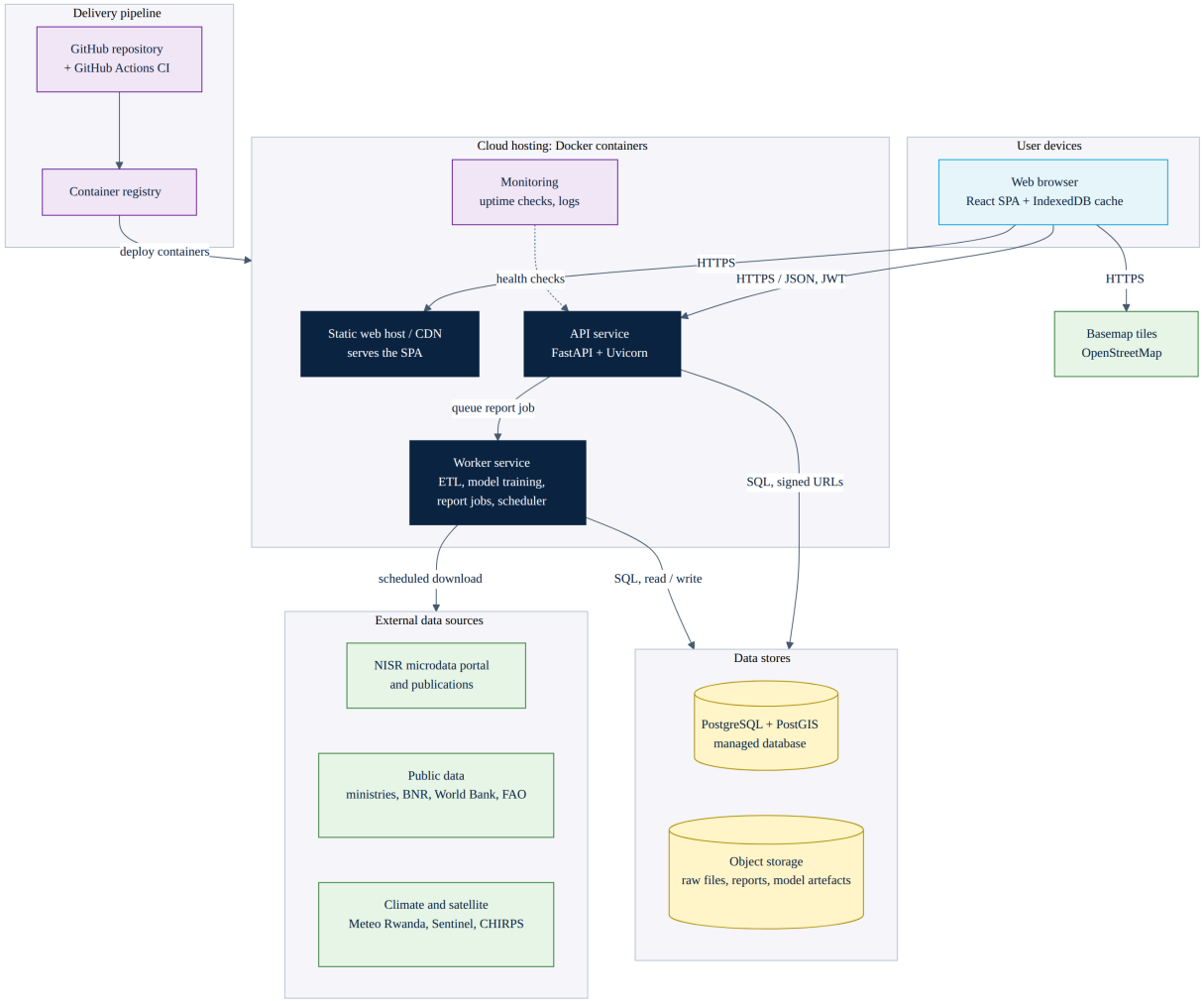


- URLs are shareable: a link to /place/nyagatare or /compare?a=nyagatare&b=kayonza reopens the exact view, which helps in reports and presentations.
- Administration pages are hidden from anyone without the administrator role.

Figure 7

System architecture

Where each part runs and how the parts communicate: the browser, the hosted services, the data stores, external data sources and the delivery pipeline.



- The browser talks only to the static host, the API and the basemap tiles; it never connects to the database or storage directly.
- Files are downloaded through short-lived signed links, so reports are never publicly listed.

Figure 8a

Sequence diagrams

Open a location profile with AI insights

The order of messages between components for the two most important interactions: opening a location profile with AI insights, and generating a report.

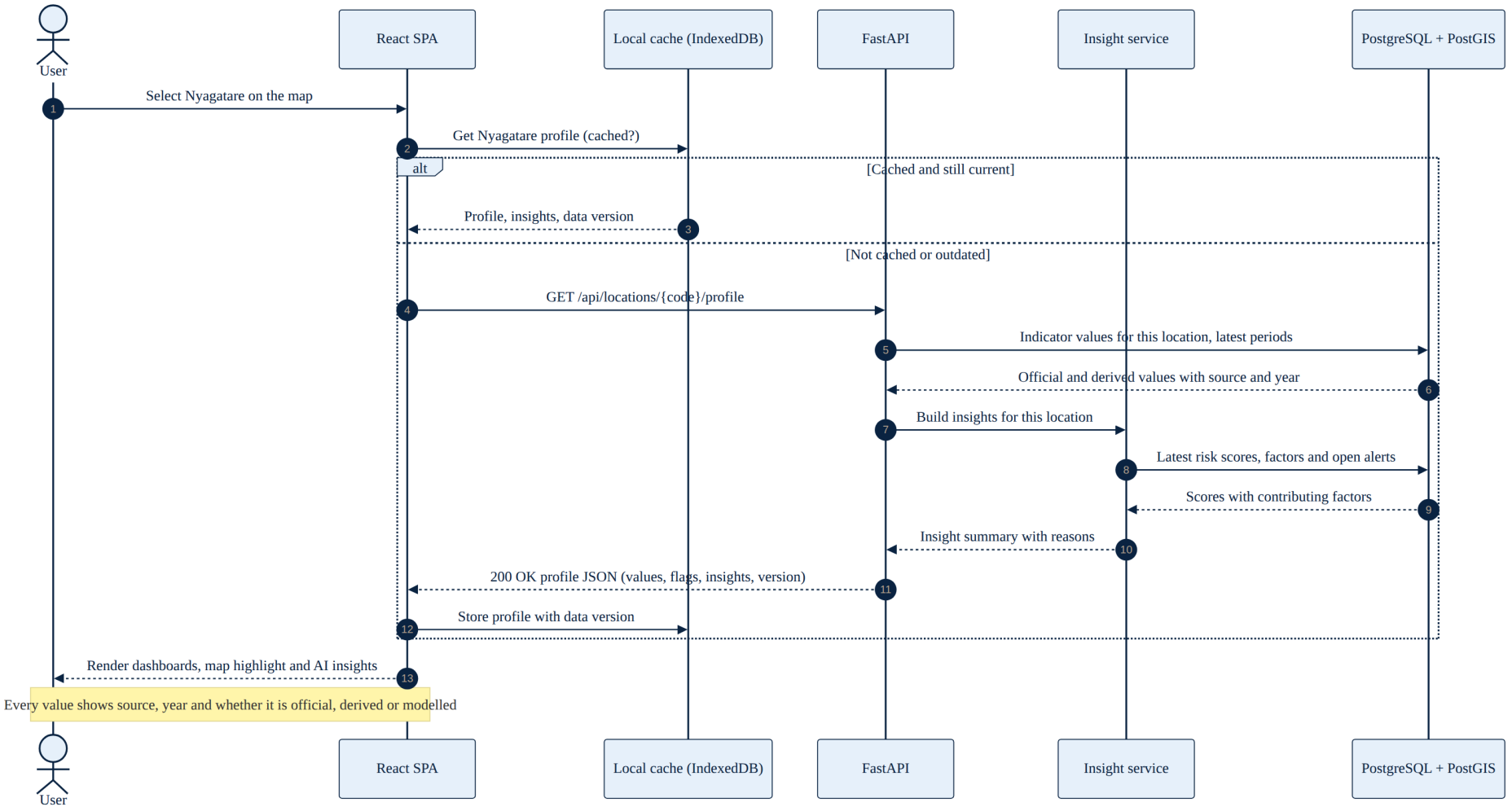
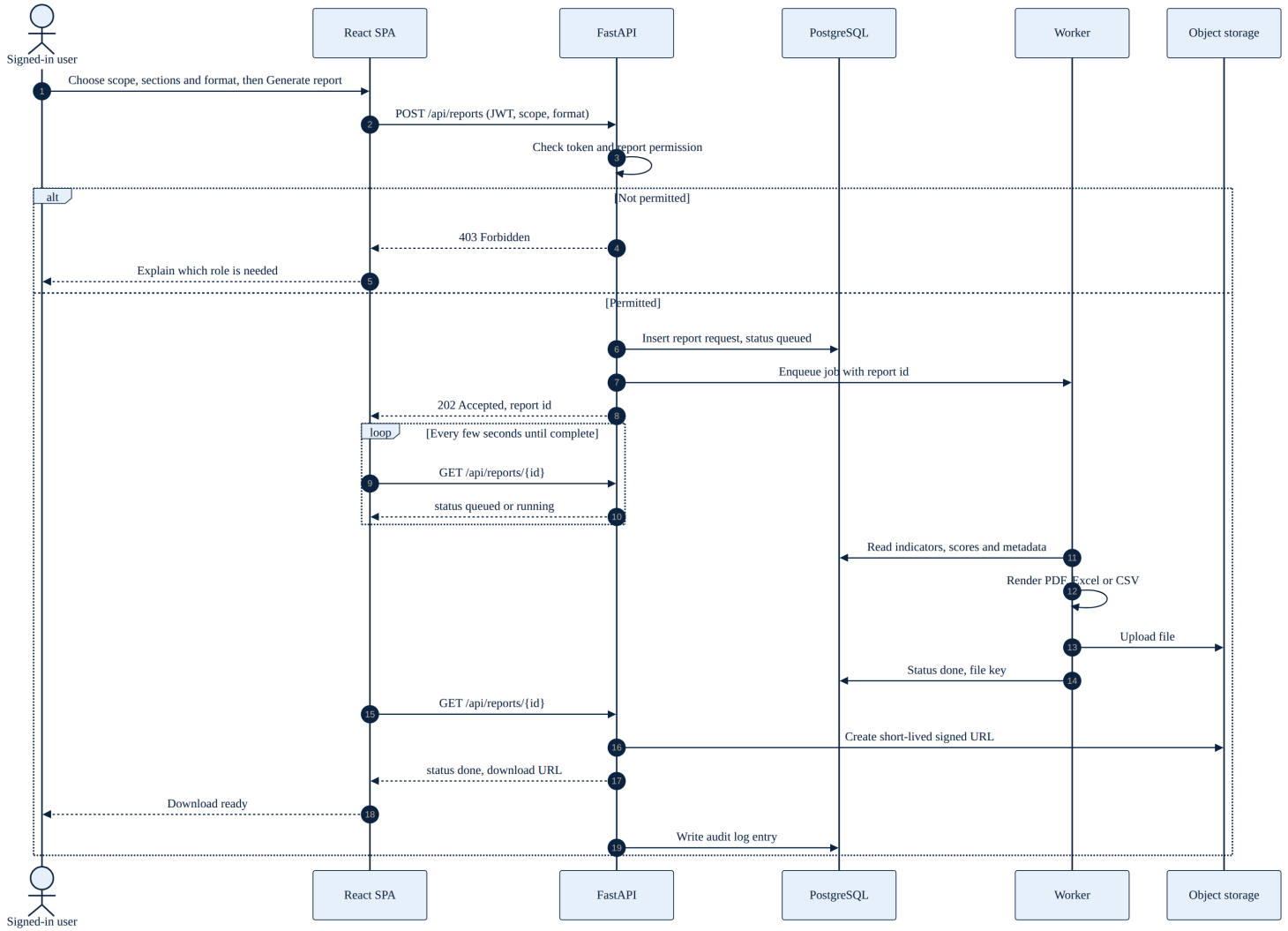


Figure 8b

Sequence diagrams

Generate and download a report



- Risk scores are calculated ahead of time by the worker, so opening a profile only reads stored results and stays fast.
- Reports are generated in the background; the page checks progress instead of waiting on one long request.

Figure 9a

Local and remote entity relationship diagrams

Remote: server database (PostgreSQL + PostGIS)

The remote ERD is the server database in PostgreSQL with PostGIS. The local ERD is the small cache kept in each user’s browser (IndexedDB) so repeat visits load instantly.

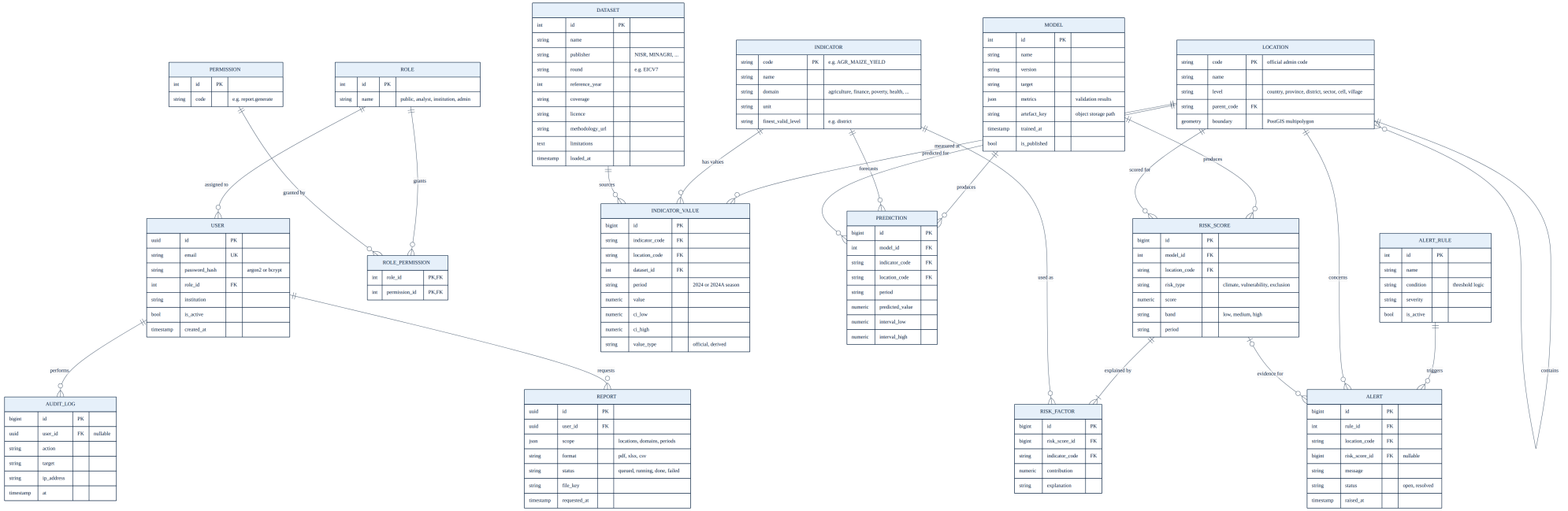
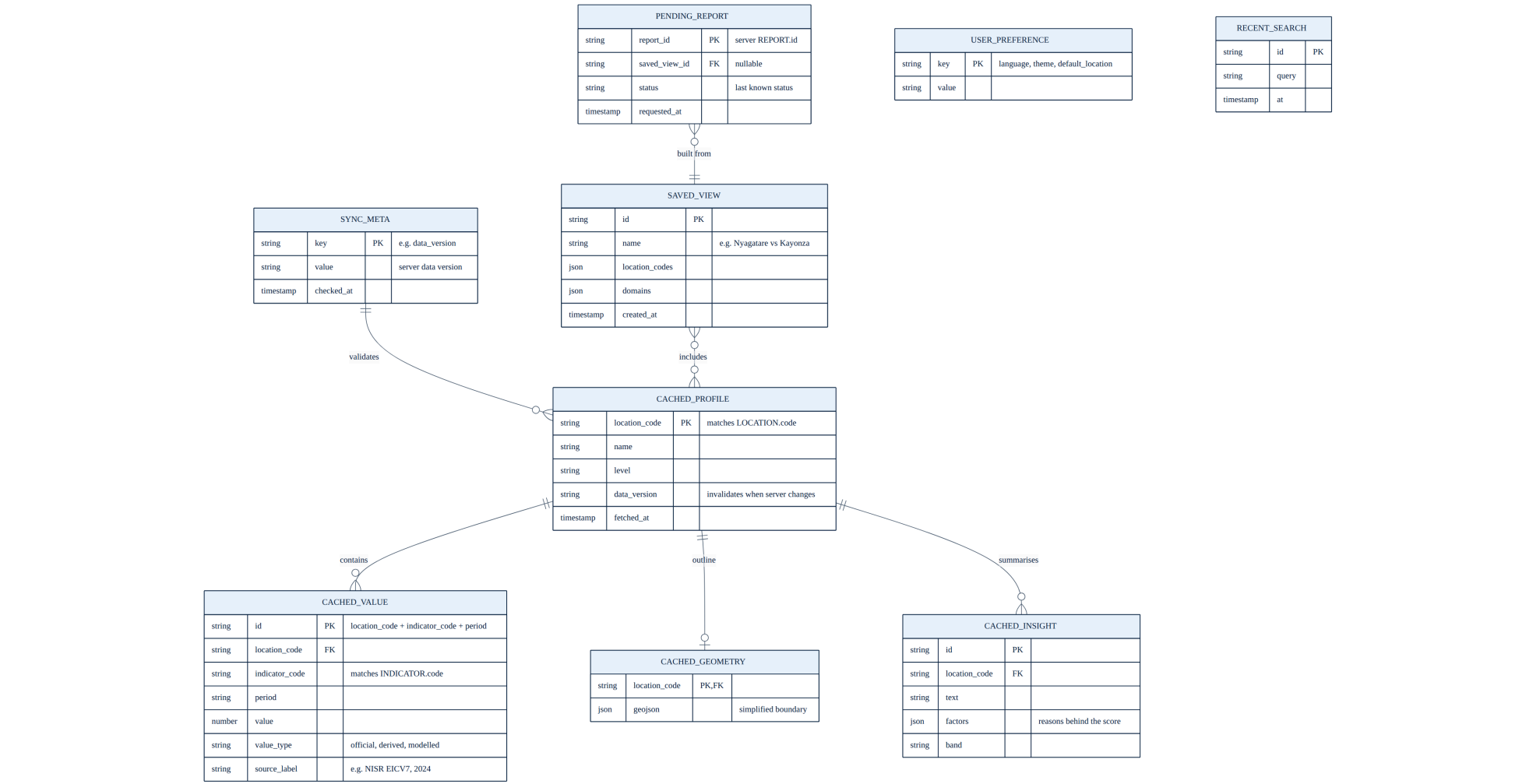


Figure 9b

Local and remote entity relationship diagrams

Local: browser cache (IndexedDB)



- All indicators share one INDICATOR_VALUE table, tagged by domain, rather than a separate table per sector. Adding a new sector then needs no schema change.
- value_type separates official statistics from derived and modelled figures, as the blueprint requires.
- The local cache holds only public, aggregate data and preferences. Sign-in tokens are never stored there.